TOP 30 CODING PROBLEMS
TREE & GRAPH

@aman_views

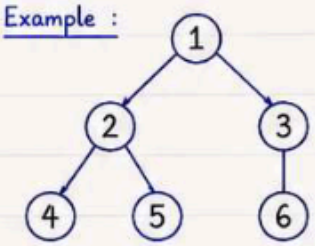
PART 1 : TREE QUESTIONS (1-3)

1. Maximum Depth of Binary Tree

Difficulty : Easy | Asked at : Amazon, Microsoft, TCS, Infosys

Problem : Find the maximum depth (height) of a binary tree.

Example :



Output : 3

Algorithm / Approach :

- If root is null, depth = 0.
- Recursively find depth of left subtree.
- Recursively find depth of right subtree.
- Answer = max(left, right) + 1.

Time Complexity : O(n) | Space : O(h)

Java Code :

```
class Solution {
    public int maxDepth(TreeNode root) {
        if (root == null) return 0;
        int left = maxDepth(root.left);
        int right = maxDepth(root.right);
        return Math.max(left, right) + 1;
    }
}
```

2. Diameter of Binary Tree

Difficulty : Medium | Asked at : Amazon, Adobe, Flipkart

Problem : Find the length of the longest path between any two nodes (may or may not pass through root).

Algorithm / Approach :

- For each node, compute height of left and right subtrees.
- Diameter at this node = leftHeight + rightHeight.
- Update global maximum diameter.
- Return height = max(leftHeight, rightHeight) + 1.

Time Complexity : O(n) | Space : O(h)

Java Code :

```
class Solution {
    int diameter = 0;
    public int diameterOfBinaryTree(TreeNode root) {
        height(root);
        return diameter;
    }
    private int height(TreeNode node) {
        if (node == null) return 0;
        int left = height(node.left);
        int right = height(node.right);
        diameter = Math.max(diameter, left + right);
        return Math.max(left, right) + 1;
    }
}
```

3. Balanced Binary Tree

Difficulty : Easy-Medium | Asked at : Microsoft, Amazon

Problem : Check if a binary tree is height-balanced (heights of two subtrees never differ by more than 1).

Algorithm / Approach :

- For each node, get height of left and right.
- If any subtree returns -1, tree is not balanced.
- If |left - right| > 1, return -1.
- Else return height = max(left, right) + 1.

Time Complexity : O(n) | Space : O(h)

Java Code :

```
class Solution {
    public boolean isBalanced(TreeNode root) {
        return height(root) != -1;
    }
    private int height(TreeNode node) {
        if (node == null) return 0;
        int left = height(node.left);
        if (left == -1) return -1;
        int right = height(node.right);
        if (right == -1) return -1;
        if (Math.abs(left - right) > 1) return -1;
        return Math.max(left, right) + 1;
    }
}
```

TOP 30 CODING PROBLEMS
TREE & GRAPH

@aman_views

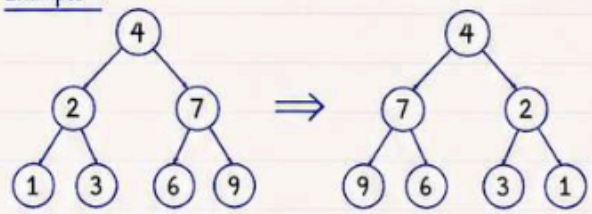
PART 1 : TREE QUESTIONS (4-6)

4. Invert Binary Tree

Difficulty : Easy | Asked at : Google, Amazon

Problem : Mirror the binary tree (swap left/right at every node).

Example :



Algorithm / Approach :

- If root is null, return null.
- Swap left and right child of current node.
- Recursively invert left and right subtrees.
- Return root.

Time Complexity : O(n) | Space : O(h)

Java Code :

```
class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) return null;
        TreeNode temp = root.left;
        root.left = invertTree(root.right);
        root.right = invertTree(temp);
        return root;
    }
}
```

---

5. Level Order Traversal (BFS)

Difficulty : Medium | Asked at : Almost every company

Problem : Return node values level by level.

Algorithm / Approach :

- If root is null, return empty list.
- Use a queue for BFS.
- For each level, process all nodes and collect values.
- Add left and right children to queue.
- Repeat until queue is empty.

Time Complexity : O(n) | Space : O(n)

Java Code :

```
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.add(level);
        }
        return result;
    }
}
```

---

6. Zigzag Level Order Traversal

Difficulty : Medium | Asked at : Amazon, Microsoft

Problem : Level order but alternate direction (left-to-right, then right-to-left).

Algorithm / Approach :

- Use BFS to traverse level by level.
- If left-to-right, add at end.
- Else add at front (use LinkedList).
- Toggle the direction after each level.

Time Complexity : O(n) | Space : O(n)

Java Code :

```
class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);
        boolean leftToRight = true;
        while (!queue.isEmpty()) {
            int size = queue.size();
            LinkedList<Integer> level = new LinkedList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (leftToRight) level.addLast(node.val);
                else level.addFirst(node.val);
                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.add(level);
            leftToRight = !leftToRight;
        }
        return result;
    }
}
```

TOP 30 CODING PROBLEMS
TREE & GRAPH

@aman_views

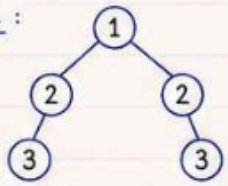
PART 1 : TREE QUESTIONS (7-9)

7. Symmetric Tree

Difficulty : Easy-Medium | Asked at : Amazon, Adobe

Problem : Check whether a tree is a mirror of itself.

Example :



Output : true

Algorithm / Approach :

- If root is null, return true.
- Check if left and right subtrees are mirror.
- Two trees are mirror if:
  - Their root values are same
  - Left of one is mirror of right of other
  - Right of one is mirror of left of other

Time Complexity : O(n) | Space : O(h)

Java Code :

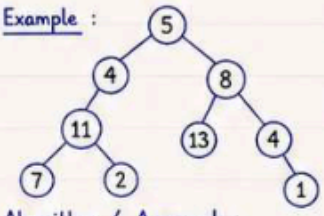
```
class Solution {
    public boolean isSymmetric(TreeNode root) {
        if (root == null) return true;
        return isMirror(root.left, root.right);
    }
    private boolean isMirror(TreeNode a, TreeNode b) {
        if (a == null && b == null) return true;
        if (a == null || b == null) return false;
        return a.val == b.val
            && isMirror(a.left, b.right)
            && isMirror(a.right, b.left);
    }
}
```

8. Path Sum

Difficulty : Easy-Medium | Asked at : Amazon, TCS, Wipro

Problem : Check if there exists a root-to-leaf path with given sum.

Example :



Target Sum = 22

Output : true

Algorithm / Approach :

- If root is null, return false.
- If it's a leaf node, check if remaining sum == node.val.
- Else, check for left and right subtrees with (targetSum - node.val).

Time Complexity : O(n) | Space : O(h)

Java Code :

```
class Solution {
    public boolean hasPathSum(TreeNode root, int targetSum) {
        if (root == null) return false;
        if (root.left == null && root.right == null)
            return targetSum == root.val;
        return hasPathSum(root.left, targetSum - root.val)
            || hasPathSum(root.right, targetSum - root.val);
    }
}
```

9. Path Sum II (All Root-to-Leaf Paths)

Difficulty : Medium | Asked at : Amazon, Microsoft

Problem : Return all root-to-leaf paths where sum of nodes == targetSum.

Example : (Same tree as above) Target Sum = 22

Output : [5,4,11,2], [5,8,4,5]

Algorithm / Approach :

- Use DFS with backtracking.
- Keep adding node values to current path.
- If leaf node and remaining sum == node.val, add a copy of path to result.
- Backtrack (remove last node) while returning.

Time Complexity : $O(n^2)$ worst case | Space : O(h)

Java Code :

```
class Solution {
    public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
        List<List<Integer>> result = new ArrayList<>();
        dfs(root, targetSum, new ArrayList<>(), result);
        return result;
    }
    private void dfs(TreeNode node, int remaining,
                     List<Integer> path, List<List<Integer>> result) {
        if (node == null) return;
        path.add(node.val);
        if (node.left == null && node.right == null && remaining == node.val) {
            result.add(new ArrayList<>(path));
        } else {
            dfs(node.left, remaining - node.val, path, result);
            dfs(node.right, remaining - node.val, path, result);
        }
        path.remove(path.size() - 1);   // backtrack
    }
}
```

TOP 30 CODING PROBLEMS
TREE & GRAPH

@aman_views

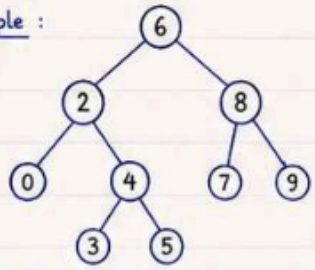
PART 1 : TREE QUESTIONS (10-12)

10. Lowest Common Ancestor of a BST

Difficulty : Easy-Medium | Asked at : Amazon, Microsoft, Flipkart

Problem : Find the lowest common ancestor (LCA) of two nodes in a Binary Search Tree.

Example :



p = 2, q = 8

Output : 6

Algorithm / Approach :

- Start from root.
- If both nodes are smaller → go left.
- If both nodes are larger → go right.
- Otherwise, current node is LCA.

Time Complexity : O(h) | Space : O(1)

Java Code :

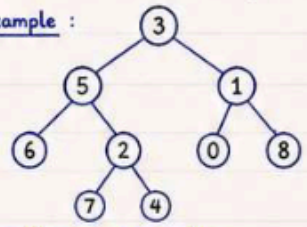
```
class Solution {
  public TreeNode lowestCommonAnecstor(TreeNode root,
                                       TreeNode p, TreeNode q) {
    TreeNode curr = root;
    while (curr != null) {
      if (p.val < curr.val && q.val < curr.val)
          curr = curr.left;
      else if (p.val > curr.val && q.val > curr.val)
          curr = curr.right;
      else
          return curr;
    }
    return null;
  }
}
```

11. Lowest Common Ancestor of a Binary Tree

Difficulty : Medium | Asked at : Amazon, Google, Microsoft

Problem : Find the LCA of two nodes in a binary tree (not necessarily a BST).

Example :



p = 5, q = 1

Output : 3

Algorithm / Approach :

- If root is null → return null.
- If root is p or q → return root.
- Recurse for left and right.
- If both sides return non-null → root is LCA.
- Else return non-null side.

Time Complexity : O(n) | Space : O(h)

Java Code :

```
class Solution {
  public TreeNode lowestCommonAnecstor(TreeNode root,
                                       TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q)
      return root;

    TreeNode left = lowestCommonAnecstor(root.left, p, q);
    TreeNode right = lowestCommonAneestor(root.right, p, q);

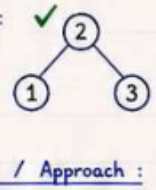
    if (left != null && right != null) return root;
    return  (left != null) ? left : right;
  }
}
```

12. Validate Binary Search Tree

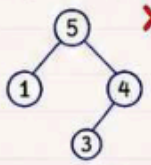
Difficulty : Medium | Asked at : Amazon, Microsoft, Adobe

Problem : Check whether a binary tree is a valid BST.

Example : ✓



✗



Algorithm / Approach :

- Use DFS with valid range (lower, upper).
- For each node, val must be in (lower, upper).
- Left subtree → upper = node.val
- Right subtree → lower = node.val
- If any node violates range → invalid BST.

Time Complexity : O(n) | Space : O(h)

Java Code :

```
class Solution {
  public boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
  }
  private boolean validate(TreeNode node, Long lower, Long upper) {
    if (node == null) return true;
    if (lower != null && node.val <= lower) return false;
    if (upper != null && node.val >= upper) return false;

    return validate(node.left, lower, (long) node.val) &&
           validate(node.right, (long) node.val, upper);
  }
}
```

TOP 30 CODING PROBLEMS
TREE & GRAPH

@aman_views

PART 2 : TREE QUESTIONS (13-15)

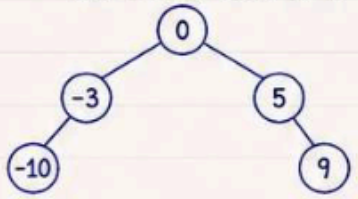
13. Convert Sorted Array to BST

Difficulty : Easy-Medium | **Asked at** : Amazon, Microsoft

Problem : Given a sorted array, convert it to a height-balanced BST.

Example :

nums = [-10, -3, 0, 5, 9]



Output : Height Balanced BST

Algorithm / Approach :

- Pick middle element as root.
- Build left subtree from left half.
- Build right subtree from right half.
- Recursively repeat.

Time Complexity : O(n) | **Space** : O(log n)

Java Code :

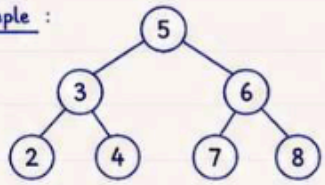
```
class Solution {
    public TreeNode sortedArrayToBST(int[] nums) {
        return build(nums, 0, nums.length - 1);
    }
    private TreeNode build(int[] nums, int left, int right) {
        if (left > right) return null;
        int mid = left + (right - left) / 2;
        TreeNode node = new TreeNode(nums[mid]);
        node.left = build(nums, left, mid - 1);
        node.right = build(nums, mid + 1, right);
        return node;
    }
}
```

14. Kth Smallest Element in a BST

Difficulty : Medium | **Asked at** : Amazon, Microsoft, Flipkart

Problem : Find the kth smallest element in a BST.

Example :



k = 3

Output : 4

Algorithm / Approach :

- Inorder traversal of BST gives sorted order.
- Count nodes during traversal.
- When count == k, return the node value.

Time Complexity : O(h + k) | **Space** : O(h)

Java Code :

```
class Solution {
    public int kthSmallest(TreeNode root, int k) {
        Deque<TreeNode> stack = new ArrayDeque<>();
        TreeNode curr = root;
        while (curr != null || !stack.isEmpty()) {
            while (curr != null) {
                stack.push(curr);
                curr = curr.left;
            }
            curr = stack.pop();
            k--;
            if (k == 0) return curr.val;
            curr = curr.right;
        }
        return -1;
    }
}
```

15. Serialize and Deserialize Binary Tree (☆ HARD)

Difficulty : Hard | **Asked at** : Amazon, Google, Microsoft

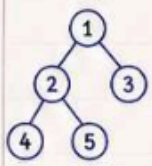
Problem : Design an algorithm to serialize a binary tree to a string and deserialize it back.

Approach :

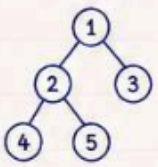
- Use preorder traversal.
- Use '#' to represent null.
- During deserialize, rebuild tree using queue.

Example :

Tree → Serialized → Deserialized



1,2,4,#,#,5,#,#,
3,#,#,



Time Complexity : O(n) | **Space** : O(n)

Java Code :

```
public class Codec {
    // Serialize
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        buildString(root, sb);
        return sb.toString();
    }
    private void buildString(TreeNode node, StringBuilder sb) {
        if (node == null) { sb.append("#,"); return; }
        sb.append(node.val).append(",");
        buildString(node.left, sb);
        buildString(node.right, sb);
    }
    // Deserialize
    public TreeNode deserialize(String data) {
        Queue<String> nodes = new LinkedList<>(Arrays.asList(data.split(",")));
        return buildTree(nodes);
    }
    private TreeNode buildTree(Queue<String> nodes) {
        String val = nodes.poll();
        if (val.equals("#")) return null;
        TreeNode node = new TreeNode(Integer.parseInt(val));
        node.left = buildTree(nodes);
        node.right = buildTree(nodes);
        return node;
    }
}
```

TOP 30 CODING PROBLEMS
TREE & GRAPH

@aman_views

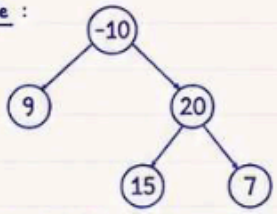
PART 2 : TREE QUESTIONS (16-18)

16. Binary Tree Maximum Path Sum (☆ HARD)

Difficulty : Hard | Asked at : Amazon, Google, Microsoft

Problem : Find the maximum path sum in a binary tree. (Path can start and end at any two nodes.)

Example :



Output : 42

(15 → 20 → 7)

Algorithm / Approach :

- Use DFS to compute max gain from left and right.
- At each node, update global max with path through this node: node.val + leftGain + rightGain.
- Return node.val + max(leftGain, rightGain) to parent.
- Ignore negative gains (use 0).

Time Complexity : O(n) | Space : O(h)

Java Code :

```
class Solution {
    int maxSum = Integer.MIN_VALUE;
    public int maxPathSum(TreeNode root) {
        maxGain(root);
        return maxSum;
    }
    private int maxGain(TreeNode node) {
        if (node == null) return 0;
        int leftGain = Math.max(maxGain(node.left), 0);
        int rightGain = Math.max(maxGain(node.right), 0);
        int priceNewPath = node.val + leftGain + rightGain;
        maxSum = Math.max(maxSum, priceNewPath);
        return node.val + Math.max(leftGain, rightGain);
    }
}
```

17. Construct Binary Tree from Preorder and Inorder Traversal

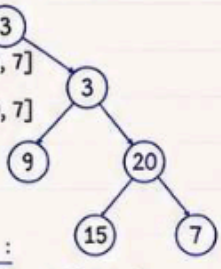
Difficulty : Medium | Asked at : Amazon, Microsoft, Adobe

Problem : Rebuild the binary tree given its preorder and inorder traversal.

Example :

Preorder = [3, 9, 20, 15, 7]

Inorder = [9, 3, 15, 20, 7]



Algorithm / Approach :

- First element in preorder is the root.
- Find root in inorder → left part is left subtree, right part is right subtree.
- Recursively build left and right subtrees.
- Use a map for inorder index lookup.

Time Complexity : O(n) | Space : O(n)

Java Code :

```
class Solution {
    Map<Integer, Integer> inorderIndex = new HashMap<>();
    int preIndex = 0;

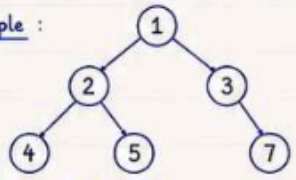
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        for (int i = 0; i < inorder.length; i++)
            inorderIndex.put(inorder[i], i);
        return build(preorder, 0, inorder.length - 1);
    }
    private TreeNode build(int[] preorder, int left, int right) {
        if (left > right) return null;
        int rootVal = preorder[preIndex++];
        TreeNode root = new TreeNode(rootVal);
        int mid = inorderIndex.get(rootVal);
        root.left = build(preorder, left, mid - 1);
        root.right = build(preorder, mid + 1, right);
        return root;
    }
}
```

18. Right Side View of Binary Tree

Difficulty : Medium | Asked at : Amazon, Microsoft, Adobe

Problem : Return the values visible from the right side of the tree, from top to bottom.

Example :



Output : [1, 3, 7]

Algorithm / Approach :

- Perform level order traversal (BFS).
- For each level, the last node popped is the rightmost.
- Add that node value to result.

Time Complexity : O(n) | Space : O(n)

Java Code :

```
class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (i == size - 1) result.add(node.val); // last node
                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
        }
        return result;
    }
}
```

TOP 30 CODING PROBLEMS
TREE & GRAPH

@aman_views

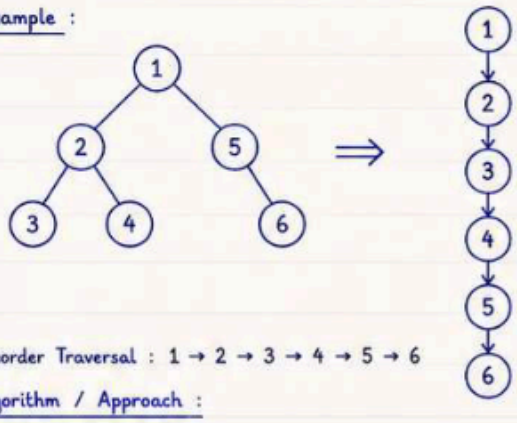
PART 2 : TREE QUESTIONS (19-20)

19. Flatten Binary Tree to Linked List

Difficulty : Medium | Asked at : Amazon, Microsoft

Problem : Flatten the binary tree into a "linked list" in-place using right pointers. The order should be preorder traversal.

Example :



Preorder Traversal : 1 → 2 → 3 → 4 → 5 → 6

Algorithm / Approach :

- Traverse the tree.
- For each node, if left child exists:
  - Find the rightmost node in left subtree.
  - Connect its right to current node's right.
  - Move left subtree to right and set left = null.
- Move to right child.
- Continue until the end.

Time Complexity : O(n) | Space : O(1)

Java Code :

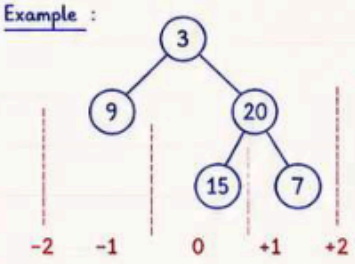
```
class Solution {
    public void flatten(TreeNode root) {
        TreeNode curr = root;
        while (curr != null) {
            if (curr.left != null) {
                // Find rightmost node in left subtree
                TreeNode rightmost = curr.left;
                while (rightmost.right != null)
                    rightmost = rightmost.right;
                // Connect rightmost to current right
                rightmost.right = curr.right;
                // Move left to right and set left null
                curr.right = curr.left;
                curr.left = null;
            }
            curr = curr.right;
        }
    }
}
```

20. Vertical Order Traversal of a Binary Tree

Difficulty : Medium | Asked at : Amazon, Microsoft, Flipkart

Problem : Group the nodes of a binary tree by vertical columns. For the same column and row, return nodes in ascending order.

Example :



Output :

[[9], [3,15], [20], [7]]

Algorithm / Approach :

- Use BFS.
- Track column index for each node.
- Use TreeMap<column, TreeMap<row, List<values>>> to keep everything sorted.
- After traversal, collect values column by column.
- For same (row, col), sort values before adding.

Time Complexity : O(n log n) | Space : O(n)

Java Code :

```
class Solution {
  public List<List<Integer>> verticalTraversal(TreeNode root) {
    // column -> row -> list of values
    TreeMap<Integer, TreeMap<Integer, List<Integer>>> map =
        new TreeMap<>();
    Queue<TreeNode> nodeQ = new LinkedList<>();
    Queue<int[]> posQ = new LinkedList<>();  // {col, row}
    nodeQ.offer(root);
    posQ.offer(new int[]{0, 0});

    while (!nodeQ.isEmpty()) {
      TreeNode node = nodeQ.poll();
      int[] pos = posQ.poll();
      int col = pos[0], row = pos[1];

      map.computeIfAbsent(col, k -> new TreeMap<>())
         .computeIfAbsent(row, k -> new ArrayList<>())
         .add(node.val);

      if (node.left != null) {
        nodeQ.offer(node.left);
        posQ.offer(new int[]{col - 1, row + 1});
      }
      if (node.right != null) {
        nodeQ.offer(node.right);
        posQ.offer(new int[]{col + 1, row + 1});
      }
    }
    List<List<Integer>> result = new ArrayList<>();
    for (TreeMap<Integer, List<Integer>> rowMap : map.values()) {
      List<Integer> colList = new ArrayList<>();
      for (List<Integer> vals : rowMap.values()) {
        Collections.sort(vals);    // sort values at same row+col
        colList.addAll(vals);
      }
      result.add(colList);
    }
    return result;
  }
```

TOP 30 CODING PROBLEMS
TREE & GRAPH

@aman_views

PART 2 : GRAPH QUESTIONS (21-23)

21. Number of Islands

Difficulty : Medium | Asked at : Amazon, Microsoft, Google

Problem : Count connected groups of "1"s in a 2D grid.

Example :

1	1	0	0	0
1	1	0	0	1
0	0	1	0	1
0	0	0	1	1

Output : 3

Algorithm / Approach :

- Traverse every cell in the grid.
- When a "1" is found, it is a new island.
- Run DFS (or BFS) to mark all connected "1"s as visited (change to "0").
- Increment island count.
- Continue the process.

Time Complexity : O(rows × cols)
Space Complexity : O(rows × cols)

Java Code :

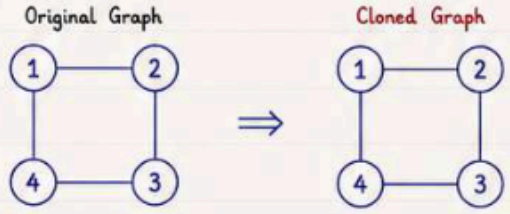
```
class Solution {
    public int numIslands(char[][] grid) {
        if (grid == null || grid.length == 0) return 0;
        int rows = grid.length, cols = grid[0].length;
        int count = 0;
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (grid[i][j] == '1') {
                    count++;
                    dfs(grid, i, j);
                }
            }
        }
        return count;
    }
    private void dfs(char[][] grid, int i, int j) {
        if (i < 0 || i >= grid.length ||
            j < 0 || j >= grid[0].length ||
            grid[i][j] != '1') return;
        grid[i][j] = '0';  // mark visited
        dfs(grid, i + 1, j);
        dfs(grid, i - 1, j);
        dfs(grid, i, j + 1);
        dfs(grid, i, j - 1);
    }
}
```

22. Clone Graph

Difficulty : Medium | Asked at : Amazon, Microsoft, Meta

Problem : Deep-copy a connected undirected graph.

Example :



Algorithm / Approach :

- Use DFS (or BFS) with a HashMap.
- Map original node → cloned node.
- For each neighbor, recursively clone and connect.
- Return the cloned starting node.

Time Complexity : O(V + E)
Space Complexity : O(V)

Java Code :

```
class Node {
    public int val;
    public List<Node> neighbors;
    public Node(int val) {
        this.val = val;
        neighbors = new ArrayList<>();
    }
}
```

```
class Solution {
    Map<Node, Node> visited = new HashMap<>();
    public Node cloneGraph(Node node) {
        if (node == null) return null;
        if (visited.containsKey(node)) return visited.get(node);
        Node clone = new Node(node.val);
        visited.put(node, clone);
        for (Node neighbor : node.neighbors) {
            clone.neighbors.add(cloneGraph(neighbor));
        }
        return clone;
    }
}
```

TOP 30 CODING PROBLEMS
TREE & GRAPH

@aman_views

PART 2 : GRAPH QUESTIONS (23-24)

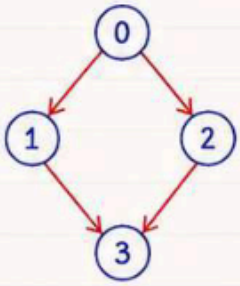
23. Course Schedule (Cycle Detection / Topological Sort)

Difficulty : Medium | **Asked at :** Amazon, Microsoft, Google

Problem : Given prerequisites, determine if you can finish all courses (i.e., no cycle).

Example :

prerequisites =
[[1, 0],
[2, 0],
[3, 1],
[3, 2]]



Output : true
(no cycle)

Algorithm / Approach :

- Build adjacency list and indegree array.
- Add all nodes with indegree 0 to the queue.
- Process nodes using BFS, reduce indegree of neighbors.
- If we visit all nodes → no cycle → can finish.
- If visited nodes < total nodes → cycle exists.

Time Complexity : O(V + E)
Space Complexity : O(V + E)

Java Code :

```
class Solution {
  public boolean canFinish(int numCourses, int[][] prerequisites) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
    int[] indegree = new int[numCourses];

    for (int[] pre : prerequisites) {
        adj.get(pre[1]).add(pre[0]);   // edge pre[1] -> pre[0]
        indegree[pre[0]]++;
    }
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++)
       if (indegree[i] == 0) queue.offer(i);

    int visited = 0;
    while (!queue.isEmpty()) {
       int curr = queue.poll();
       visited++;
       for (int next : adj.get(curr)) {
           indegree[next]--;
           if (indegree[next] == 0) queue.offer(next);
       }
    }
    return visited == numCourses;   // no cycle if all visited
  }
}
```

---

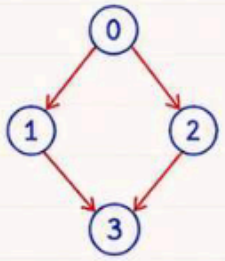
24. Course Schedule II (Return Valid Order)

Difficulty : Medium | **Asked at :** Amazon, Microsoft

Problem : Same as above but return the actual valid course order.

Example :

prerequisites =
[[1, 0],
[2, 0],
[3, 1],
[3, 2]]



Output :
[0, 1, 2, 3]
(one possible order)

Algorithm / Approach :

- Use Kahn's Algorithm (BFS Topological Sort).
- Build adjacency list and indegree array.
- Push all nodes with indegree 0 to queue.
- Pop node, add to order, reduce indegree of neighbors.
- If order size == numCourses → return order.
- Else → cycle exists → return empty array.

Time Complexity : O(V + E)
Space Complexity : O(V + E)

Java Code :

```
class Solution {
  public int[] findOrder(int numCourses, int[][] prerequisites) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
    int[] indegree = new int[numCourses];

    for (int[] pre : prerequisites) {
        adj.get(pre[1]).add(pre[0]);   // edge pre[1] -> pre[0]
        indegree[pre[0]]++;
    }
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++)
       if (indegree[i] == 0) queue.offer(i);

    int[] order = new int[numCourses];
    int idx = 0;
    while (!queue.isEmpty()) {
       int curr = queue.poll();
       order[idx++] = curr;
       for (int next : adj.get(curr)) {
           indegree[next]--;
           if (indegree[next] == 0) queue.offer(next);
       }
    }
    return idx == numCourses ? order : new int[0];
  }
}
```

TOP 30 CODING PROBLEMS
TREE & GRAPH

@aman_views

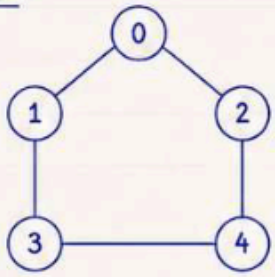
PART 2 : GRAPH QUESTIONS (25-26)

25. Detect Cycle in an Undirected Graph

Difficulty : Medium | Asked at : Amazon, Microsoft, Service-based companies

Problem : Check if an undirected graph contains a cycle.

Example :



Output : true (cycle exists)

Algorithm / Approach :

- Use DFS.
- Keep track of parent node.
- If we visit an already visited node that is not the parent, it means a cycle exists.

Time Complexity : O(V + E)
Space Complexity : O(V)

Java Code :

```
class Solution {
  public boolean hasCycle(int V, List<List<Integer>> adj) {
    boolean[] visited = new boolean[V];
    for (int i = 0; i < V; i++) {
      if (!visited[i]) {
        if (dfs(i, -1, visited, adj)) return true;
      }
    }
    return false;
  }
  private boolean dfs(int node, int parent,
                      boolean[] visited, List<List<Integer>> adj) {
    visited[node] = true;
    for (int neighbor : adj.get(node)) {
      if (!visited[neighbor]) {
        if (dfs(neighbor, node, visited, adj)) return true;
      }
      else if (neighbor != parent) {
        // visited neighbor is not parent -> cycle
        return true;
      }
    }
    return false;
  }
}
```

26. Rotting Oranges (Multi-Source BFS)

Difficulty : Medium | Asked at : Amazon, Microsoft, Flipkart

Problem : Every minute, rotten oranges rot adjacent fresh oranges. Find minimum minutes until no fresh orange remains (or -1 if impossible).

Example :

Initial Grid

2	1	1
1	1	0
0	1	1

⟹

After 4 minutes

2	2	2
2	2	0
0	2	2

Output : 4

Algorithm / Approach :

- All rotten oranges (2) are added to queue initially.
- Perform BFS level by level (each level = 1 minute).
- Rotten oranges spread to adjacent fresh ones (1).
- Count fresh oranges. If any remain at the end → return -1.
- Use 4 directions: up, down, left, right.

Time Complexity : O(rows × cols)
Space Complexity : O(rows × cols)

Java Code :

```
class Solution {
  public int orangesRotting(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    Queue<int[]> queue = new LinkedList<>();
    int fresh = 0;

    // Add all rotten oranges and count fresh ones
    for (int i = 0; i < rows; i++) {
      for (int j = 0; j < cols; j++) {
        if (grid[i][j] == 2) queue.offer(new int[]{i, j});
        else if (grid[i][j] == 1) fresh++;
      }
    }
    if (fresh == 0) return 0;
    int minutes = 0;
    int[][] dirs = {{0,1}, {0,-1}, {1,0}, {-1,0}};

    while (!queue.isEmpty() && fresh > 0) {
      int size = queue.size();
      for (int k = 0; k < size; k++) {
        int[] cell = queue.poll();
        for (int[] d : dirs) {
          int r = cell[0] + d[0], c = cell[1] + d[1];
          if (r >= 0 && r < rows && c >= 0 && c < cols
              && grid[r][c] == 1) {
            grid[r][c] = 2;
            fresh--;
            queue.offer(new int[]{r, c});
          }
        }
      }
      minutes++;
    }
    return (fresh == 0) ? minutes : -1;
  }
}
```

TOP 30 CODING PROBLEMS
TREE & GRAPH

@aman_views

PART 2 : GRAPH QUESTIONS (27)

27. Alien Dictionary

Difficulty : Hard | Asked at : Amazon, Facebook, Google

Problem : Given a sorted list of words from an alien dictionary, find the order of characters. Return an empty string if invalid order.

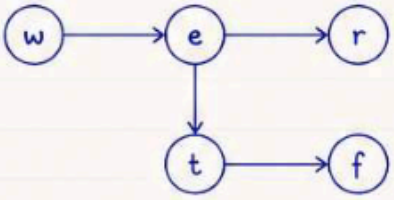
Example :

Input : words = ["wrt","wrf","er","ett","rftt"]

Output : "wertf"

wrt	Compare wrt and wrf → t < f
wrf	Compare wrf and er → w < e
er	Compare er and ett → r < t
ett	Compare ett and rftt → e < r
rftt	

Graph (Directed Edges) :



Topological Order :

w	e	r	t	f

(One Possible Answer)

Algorithm / Approach :

- Build a graph by comparing adjacent words.
- For first mismatch, add directed edge from mismatch char of first word to second word.
- Compute in-degree of all nodes.
- Use BFS (Kahn's Algorithm) to get topological order of characters.
- If order length != number of unique chars → cycle exists → return "".

Time Complexity : O(V + E)
Space Complexity : O(V + E)

Java Code :

```
class Solution {
  public String alienOrder(String[] words) {
    // Step 1: Build graph and in-degree map
    Map<Character, List<Character>> adj = new HashMap<>();
    Map<Character, Integer> indegree = new HashMap<>();

    // Add all unique characters
    for (String word : words) {
      for (char c : word.toCharArray()) {
        adj.putIfAbsent(c, new ArrayList<>());
        indegree.putIfAbsent(c, 0);
      }
    }
    // Build edges by comparing adjacent words
    for (int i = 0; i < words.length - 1; i++) {
      String w1 = words[i], w2 = words[i + 1];
      int len = Math.min(w1.length(), w2.length());
      for (int j = 0; j < len; j++) {
        char c1 = w1.charAt(j), c2 = w2.charAt(j);
        if (c1 != c2) {
          adj.get(c1).add(c2);
          indegree.put(c2, indegree.get(c2) + 1);
          break;
        }
      }
    }
    // Step 2: Topological Sort (BFS)
    Queue<Character> queue = new LinkedList<>();
    for (char c : indegree.keySet()) {
      if (indegree.get(c) == 0) queue.offer(c);
    }
    StringBuilder sb = new StringBuilder();
    while (!queue.isEmpty()) {
      char curr = queue.poll();
      sb.append(curr);
      for (char nei : adj.get(curr)) {
        indegree.put(nei, indegree.get(nei) - 1);
        if (indegree.get(nei) == 0) queue.offer(nei);
      }
    }
    // Step 3: If not all chars processed → cycle
    if (sb.length() != indegree.size()) return "";
    return sb.toString();
  }
```